Введение

План

- ❑ Взаимодействие с LLM
- ❑ Облачное API vs локальное взаимодействие
- ❑ Установка и запуск
- ❑ Интеграция с python
- ❑ Base models vs Instract models
- ❑ Эксперименты с промптами: Few-shot и Chain-of-Thought

Осознанное взаимодействие с LLM: промпты, параметры, security

❑ Ключевые параметры генерации (Hyperparameters)

- Температура. 0 — ответ детерменирован
- Top_p (Nucleus sampling):* Альтернатива температуре. Модель рассматривает только токены, чья совокупная вероятность превышает p (например, 0.9).
- Max_tokens:** Ограничение длины ответа (важно для контроля затрат и предотвращения "галлюцинаций" или бесконечных циклов).

Осознанное взаимодействие с LLM: промпты, параметры, security

Группа	Параметры	Что трогают	Эффект
Форма	`temperature`	резкость распределения	детерминизм ↔ разнообразие
Поддержка	`top_k`, `top_p`, `min_p`, `typical_p`	какие токены допущены	обрезка «хвоста»
История	`frequency_penalty`, `presence_penalty`, `repetition_penalty`, `repeat_last_n`	форма с учётом написанного	против повторов, против зацикливания
Адаптив	`mirostat`, `mirostat_tau`, `mirostat_eta`	температура *в рантайме*	ровное качество на длинных текстах
Жёсткие	`grammar` / `response_format` / JSON-schema, `logit_bias`, `stop`, `max_tokens`	пространство допустимого / завершение	гарантии формата и остановки
Детерминизм	`seed`	ГСЧ	воспроизводимость (с оговорками)
Структура	`num_ctx`, `rope_freq_base`	окно внимания	качество на длинном контексте
Только скорость	`n_batch`, `n_gpu_layers`, `flash_attn`, `num_thread`, `keep_alive`	не качество	латентность, VRAM

Выбор между облачными API и локальными моделями

Критерий	**Облачные API (OpenAI, Anthropic, YandexGPT)**	**Локальные модели (Llama 3, Mistral, Qwen через Ollama)**
Мощность	Максимальная (доступ к флагманским моделям)	Зависит от вашего железа (VRAM)
Конфиденциальность	Низкая/Средняя (данные уходят на сервер провайдера)	Высокая (данные не покидают ваш компьютер)
Стоимость	Оплата за токены (может быть дорого при масштабировании)	Бесплатно (после покупки железа)
Сложность	Низкая (один API-ключ)	Средняя (нужна настройка окружения, знание форматов)
Latency	Зависит от интернета и загрузки серверов	Минимальная (зависит от скорости GPU)

Base-модели и Instruct-модели: в чём разница и почему это важно

- **Base-модели (Предобученные)**: Обучены только на предсказании следующего слова на огромных массивах текста (Common Crawl, Wikipedia). Они не умеют "общаться". Если вы спросите у base-модели: "Столица Франции?", она может продолжить фразу: "... это Париж, а столица Германии – Берлин..." или просто выдать список столиц, потому что она видит паттерн "Столица X – это Y".
- **Instruct/Chat-модели**: Прошли дополнительный этап обучения (SFT - Supervised Fine-Tuning и RLHF/DPO). Их научили следовать инструкциям, вести диалог и форматировать вывод.

Base-модели и Instruct-модели: в чём разница и почему это важно

- Почему это важно для кода? Instruct-модели требуют специального форматирования промпта (Chat Template). Если скормить base-модели диалог в формате чата, она просто продолжит генерировать текст от имени пользователя.

Эксперименты с промптами: Few-shot и Chain-of-Thought

Промпт-инжиниринг — это способ "программировать" модель без изменения её весов.

- **Zero-shot vs Few-shot**
- Zero-shot: Мы просто задаем вопрос. Модель может ошибиться в формате.
- Few-shot: Мы даем 2-3 примера "входные данные -> ожидаемый выход".

  Это резко повышает качество и соблюдение формата.

Chain-of-Thought (CoT)

- Метод, заставляющий модель рассуждать шаг за шагом перед тем, как дать финальный ответ. Критически важен для математических и логических задач. Магическая фраза: "Давай подумаем шаг за шагом" (Let's think step by step).


LLM